

Simulazioni n -body su GPU: approccio naive e Barnes-Hut

Relazione del progetto di GPGPU - Damiano Trovato

Abstract

Con la seguente relazione si documenta la realizzazione di simulazioni n -body tramite soluzioni parallele su GPU. Si trattano quindi aspetti degni di nota, quali gli algoritmi, i metodi numerici, le ottimizzazioni e un'analisi delle performance. Le soluzioni principali presentate saranno due: un'approccio naive su GPU, e un approccio approssimato Barnes-Hut, sempre su GPU.

1 Introduzione

In questa sezione offriamo un'introduzione al progetto, dando una definizione del problema n -body, i dettagli fisici e matematici e come verranno sviluppati i test.

1.1 Simulazioni n -body

Una simulazione a n corpi permette di calcolare l'evoluzione temporale di un sistema costituito da, appunto, n corpi, che interagiscono tra loro con delle forze che dipendono dalla simulazione. Queste simulazioni sono fondamentali, in quanto ogni problema a n corpi non banale, e con non configurazione non banale¹, non dispone di una soluzione analitica.

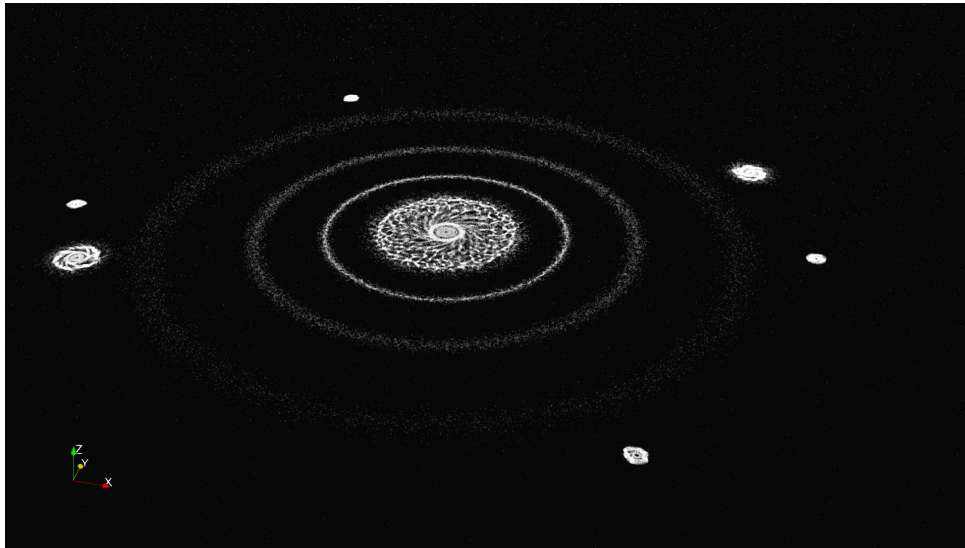


Immagine 1: Una demo a 180k corpi: <https://youtu.be/oSdXonzxpXk?si=4AL8uCfM6irI1BcW>

Sono fondamentali nell'ambito dell'astrofisica computazionale, offrendo soluzioni per osservare l'evoluzione di sistemi cosmologici, seppur simulazioni accurate devono considerare anche i gas, e quindi implementare elementi di fluidodinamica.

¹assenza totale di forze, o configurazioni con determinate simmetrie.

1.2 Aspetti fisici e matematici

Si offre una lista delle formule matematiche che rendono fattibili le simulazioni realizzate all'interno del progetto. Le seguenti formule sono state prese dal video [?].

Legge di gravitazione universale La seguente formula detta l'attrazione che agisce tra due corpi rispettivamente di massa m_1 ed m_2 , situate a distanza $|r|$. Questa grandezza è direttamente proporzionale al prodotto delle masse e inversamente proporzionale al quadrato della distanza, ossia:

$$F = G \frac{m_1 m_2}{|r|^2} \hat{r} = G \frac{m_1 m_2}{|r|^2} \cdot \frac{r}{|r|} = G \frac{m_1 m_2}{|r|^3} r. \quad (1)$$

In natura G è una costante minuscola che rende l'attrazione tra i corpi rilevante solo se i corpi hanno massa elevatissima o distanza reciproca ridotta. Ai fini della simulazione, questa costante verrà portata a valori molto più grandi. $\hat{r}$ indica invece il versore² che punta da m_1 a m_2 , ottenuto normalizzando r . Sostituiamo quindi $\hat{r}$ con la sua formula esplicita.

Dalla forza all'accelerazione Supponiamo sul corpo m_1 agisca una determinata forza F :

$$F = m_1 a \Rightarrow a = \frac{F}{m_1} = G \frac{m_2}{|r|^3} r. \quad (2)$$

In definitiva, l'accelerazione che agisce su un corpo m_i è data da:

$$a_i = \sum_{j=0, j \neq i}^{n-1} G \frac{m_j}{|r_j|^3} r_j. \quad (3)$$

Dall'accelerazione alla velocità L'accelerazione è la derivata della velocità sul tempo, possiamo quindi calcolare la velocità di un corpo tramite

$$a = \frac{dv}{dt} \Rightarrow a dt = dv, \quad (4)$$

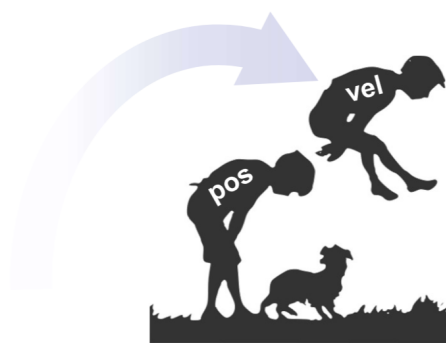
fissando un valore di $dt = \text{delta_time}$.

Dalla velocità allo spostamento

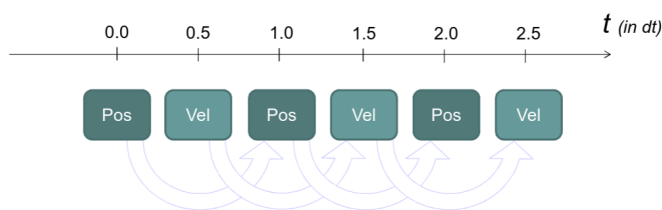
$$x = x_0 + v dt \quad (5)$$

dove $dt = \text{delta_time}$.

1.3 Metodo numerico di integrazione: leapfrog



3D video gaming - Marco Tarini
Univ Milano 2019/2020



Il metodo numerico utilizzato nella simulazione, è noto come metodo **leapfrog**, o metodo della cavallina. Il nome deriva dal modo in cui si alterna l'integrazione della velocità a quella del tempo.

Questo metodo numerico, sfasando di $dt/2$ l'integrazione della velocità e della posizione, ottiene errore ridotto, miglior comportamento asintotico e conservazione dell'energia del sistema, rispetto al metodo di Eulero. Sono tecniche di integrazione egualmente onerose, ma il metodo Leapfrog è più accurato e totalmente reversibile [?].

²vettore unitario

Pseudo-codice 1 Integrazione Leapfrog

```
1: Calcola l'accelerazione
2: Aggiorna la velocità per  $dt/2$ 
3: for Numero di iterazioni do
4:   Aggiorna la posizione per  $dt$ 
5:   Calcola l'accelerazione
6:   Aggiorna la velocità per  $dt$ 
7: end for
```

1.4 Studio delle performance su hardware

I test delle varie soluzioni fornite, saranno effettuate su tre schede video differenti. A seguire, nomi e alcuni dettagli ottenuti con `clinfo`:

- **AMD Radeon RX 6600 CLD 8GB** (discreta).
- **Intel Core Ultra 9 285H - Intel Arc Pro 130T/140T** (integrata).
- **AMD Ryzen 7 7700 - gfx1036** (integrata).

In tutte le soluzioni affrontate, verranno omesse le operazioni di scrittura a disco, che rappresentano una grande fonte di overhead, ma che sono anche fuori dallo scope del progetto.

Le nostre valutazioni avverranno rispetto ad un numero crescente di corpi n da 10 a ~ 180000 . Per ogni simulazione, valutiamo il calcolo di 10 frame, in modo da ridurre l'effetto della varianza sul singolo tempo di esecuzione del kernel, su un frame particolarmente fortunato (o sfortunato).

2 Simulazione parallela naive su GPU

Il primo approccio alla simulazione di sistemi a n corpi che si è deciso di realizzare, è una soluzione parallela basata sull'algoritmo naive, con complessità computazionale $\mathcal{O}(n^2)$. Consiste banalmente nel calcolo brute-force di tutte le interazioni corpo-a-corpo.

2.1 Kernel fondamentali e ottimizzazioni

Calcolo dell'accelerazione naive Il primo kernel che trattiamo, viene eseguito su ogni corpo per ogni istante della simulazione, e permette il calcolo dell'accelerazione su ogni corpo. dalla teoria all'implementazione,

Pseudo-codice 2 Kernel per il calcolo dell'accelerazione di un corpo

Require: array dei corpi C , $n = |C|$

```
1:  $i \leftarrow$  id globale del work-item
2: if  $i \geq n$  then return     $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
3: end if
4:  $a \leftarrow 0$ 
5: for all  $j \in \text{Range}(n)$  do
6:   if  $i = j$  then
7:     continue
8:   end if
9:    $r \leftarrow C_{pos}[i] - C_{pos}[j]$ 
10:   $r = \max(r, \varepsilon)$ 
11:   $a \leftarrow G \frac{C_{mass}[j]}{|r|^3} r + a$ 
12: end for
13:  $C_{acc}[i] \leftarrow a$ 
```

subentrano delle ottimizzazioni. Il calcolo di $\frac{1}{|r|^3}$ è infatti ottimizzato col calcolo della radice quadrata inversa su r al cubo (notoriamente più veloce del calcolo esplicito della radice quadrata e della divisione).

```
const float2 dist_vec = body_pos[i] - this_pos;
const float dist_squared = fmax(dot(dist_vec, dist_vec), EPSILON);
const float inv_dist = rsqrt(dist_squared);
const float inv_dist3 = inv_dist * inv_dist * inv_dist;
this_acc += G * body_mass[i] * dist_vec * inv_dist3;
```

Osserviamo inoltre una funzione $\max(r, \varepsilon)$, con ε valore arbitrariamente piccolo per evitare che $|r|^3 \rightarrow 0$ (con conseguente accelerazione $\rightarrow \pm\infty$) o peggio ancora $|r|^3 = 0$, che causerebbe un errore di divisione per zero. Questo problema si pone perché nella simulazione non consideriamo collisioni, i corpi possono passarsi attraverso, ottenendo valori molto piccoli di r . Scegliamo il valore ε sperimentalmente.

Calcolo della velocità Il seguente kernel è invece dedicato al calcolo della velocità dall'accelerazione. Il delta tempo dt è un argomento, in quanto necessario per il metodo Leapfrog.

Pseudo-codice 3 Kernel per il calcolo della velocità

Require: array dei corpi C , $n = |C|$, delta tempo dt

```
1:  $i \leftarrow$  id globale del work-item
2: if  $i \geq n$  then return     $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
3: end if
4:  $C_{vel}[i] \leftarrow C_{vel}[i] + C_{acc}[i] \cdot dt$ 
```

Calcolo della posizione L'ultimo kernel della simulazione naive è dedicato all'aggiornamento della posizione.

Pseudo-codice 4 Kernel per il calcolo della posizione

Require: array dei corpi C , $n = |C|$, delta tempo dt

```
1:  $i \leftarrow$  id globale del work-item  
2: if  $i >= n$  then return  $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input  
3: end if  
4:  $C_{pos}[i] \leftarrow C_{pos}[i] + C_{vel}[i] \cdot dt$ 
```

2.2 Codice host, algoritmo complessivo

Pseudo-codice 5 Simulazione n -body naive

Require: array dei corpi C , $n = |C|$, delta tempo dt , numero di iterazioni t

```
1: lancia  $n$  kernel accelerazione naive su  $C$  con delta tempo  $dt$   
2: lancia  $n$  kernel calcolo velocità su  $C$  con delta tempo  $dt/2$   
3: for  $t$  do  
4:   lancia  $n$  kernel calcolo posizione su  $C$  con delta tempo  $dt$   
5:   lancia  $n$  kernel accelerazione naive su  $C$  con delta tempo  $dt$   
6:   lancia  $n$  kernel calcolo velocità su  $C$  con delta tempo  $dt$   
7: end for
```

In realtà, per ogni kernel lanciato, il numero di kernel è quasi sempre arrotondato al primo multiplo di 32 più grande di n , per evitare che le performance vengano penalizzate (aggiungerei, in maniera disastrosa) da un numero primo di particelle.

2.3 Benchmarking

Procediamo adesso con un'analisi delle performance dell'algoritmo naive. Prima di parlare con i dati alla mano, ricapitoliamo la complessità asintotica dei singoli passaggi (con una prospettiva seriale):

- aggiornamento posizione: $\mathcal{O}(n)$;
- aggiornamento velocità: $\mathcal{O}(n)$;
- aggiornamento accelerazione: $\mathcal{O}(n^2)$;

ma anche il numero di accessi in memoria globale, che sappiamo rappresentare una delle più grandi fonti di bottleneck nell'esecuzione dei kernel, che sono:

- aggiornamento posizione: 2, uno alla velocità e uno alla posizione, $\mathcal{O}(1)$;
- aggiornamento velocità: 2, uno all'accelerazione e uno alla velocità, $\mathcal{O}(1)$;
- aggiornamento accelerazione: $1 + 2n$, n alle posizioni, n alle masse, 1 all'accelerazione, $\mathcal{O}(n)$;

e ricordando che questo vale per un numero di work-item pari al numero di corpi, ossia n . Seppur sembra un'osservazione triviale, questo numero fisso di work-item (uguale al numero di corpi) non sarà presente nell'approccio Barnes-Hut proposto.

Osservando i grafici delle successive pagine, emergono un paio di valutazioni. Entrambe le GPU integrate valutate performano in maniera simile sul calcolo di posizione e velocità. In generale, il calcolo di posizione e velocità cresce linearmente in funzione del numero di corpi.

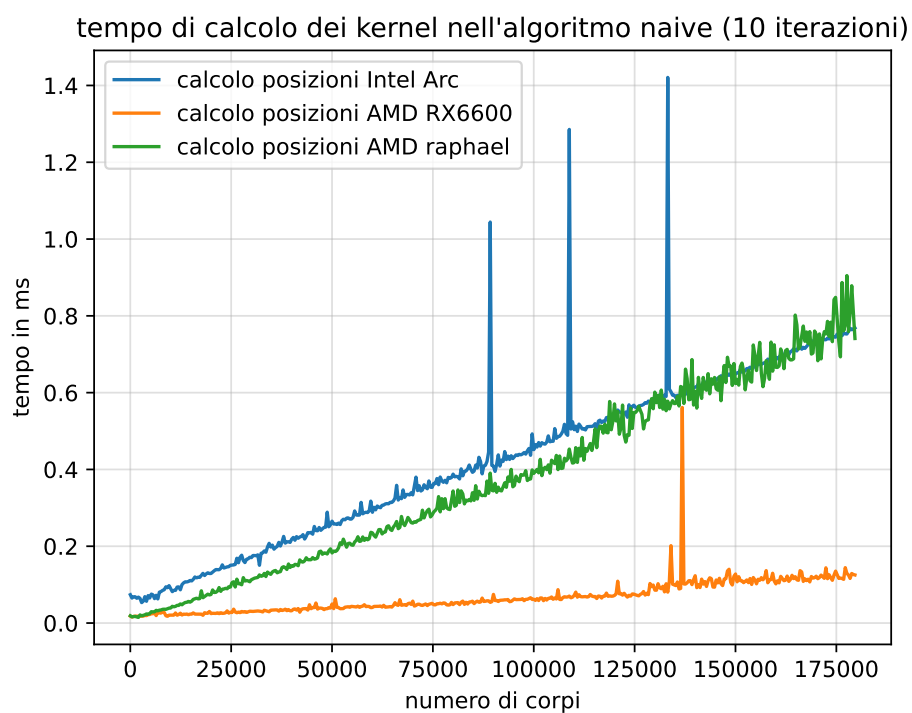


Immagine 2: Tempo di calcolo della posizione, in funzione del numero di corpi, valutato su tre schede video differenti. In arancione, abbiamo l'unica GPU discreta, la RX6600 di casa AMD.

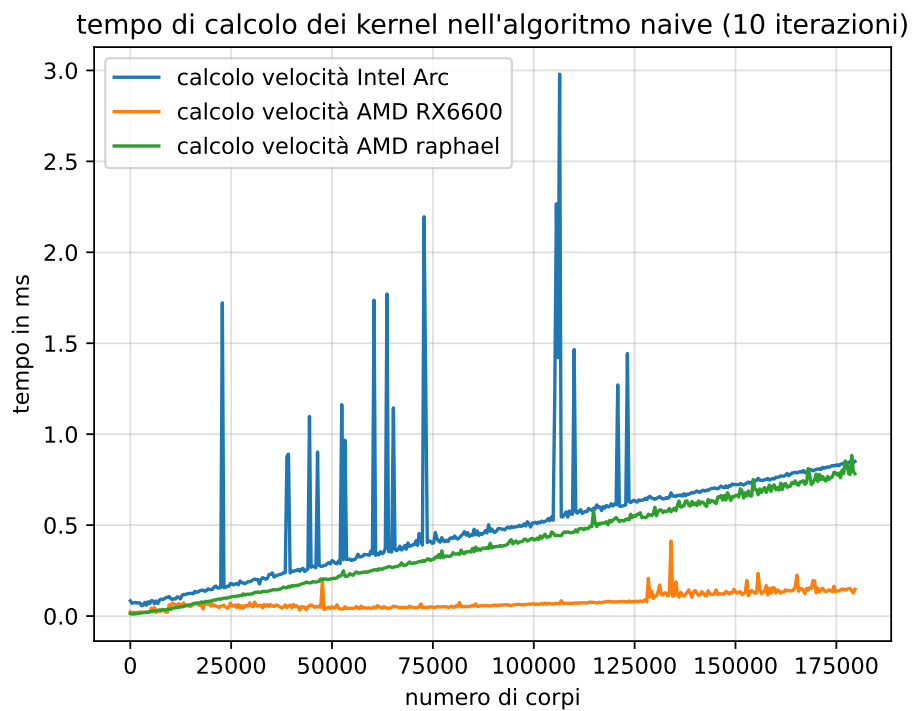


Immagine 3: Tempo di calcolo della velocità, in funzione del numero di corpi.

Meno fortunato è il calcolo dell'accelerazione dei corpi. L'alto numero di operazioni e di accessi alla memoria globale, demolisce totalmente le performance della grafica integrata del Ryzen 7 7700 (Raphael), meno avanzata se messa a confronto con le performance della grafica integrata Intel.

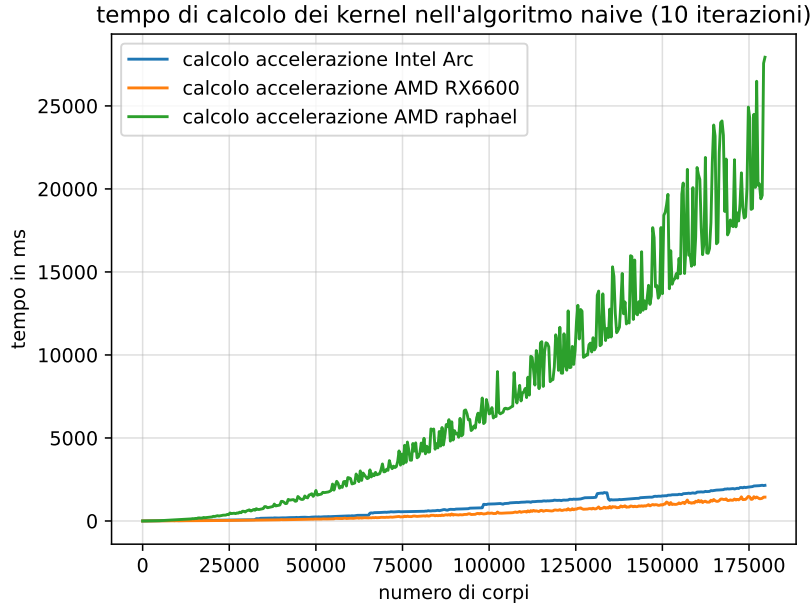


Immagine 4: Tempo di calcolo dell'accelerazione, in funzione del numero di corpi.

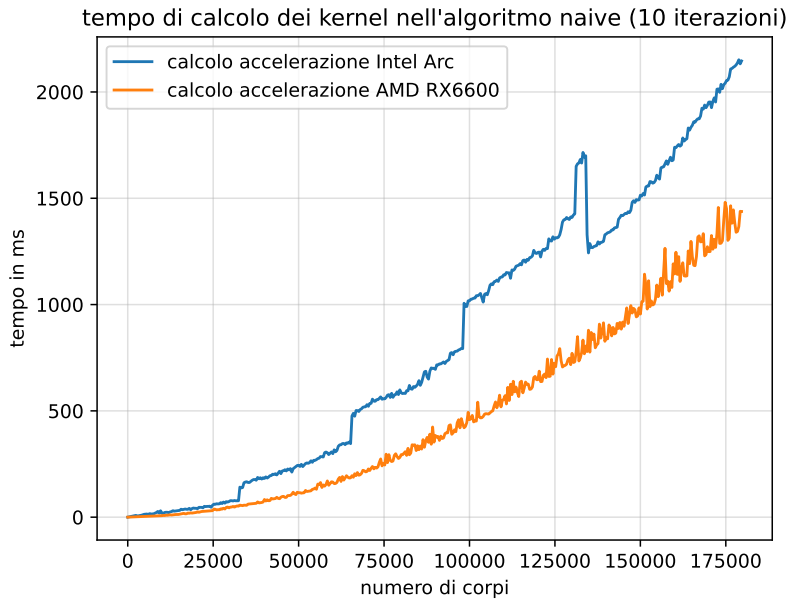


Immagine 5: Ai fini di visualizzazione, proponiamo lo stesso grafico precedente escludendo l'AMD Raphael.

Seppur tutte le visualizzazioni dei sistemi siano state effettuate in un secondo momento all'interno del software ParaView, è possibile calcolare il tempo massimo ammesso per il calcolo di un fotogramma della simulazione, in contesto realtime. La formula è

$$t = \frac{1000}{\text{frame per secondo}} \text{ ms}, \quad (6)$$

per cui, i tempi per calcolare un frame a 30 FPS dovranno essere al più $t_{30} = 33,33 \text{ ms}$, per 60 FPS al più la metà, ossia $t_{60} = 16,6 \text{ ms}$. Moltiplichiamo queste quantità per 10, per poter riutilizzare i dati già elaborati.

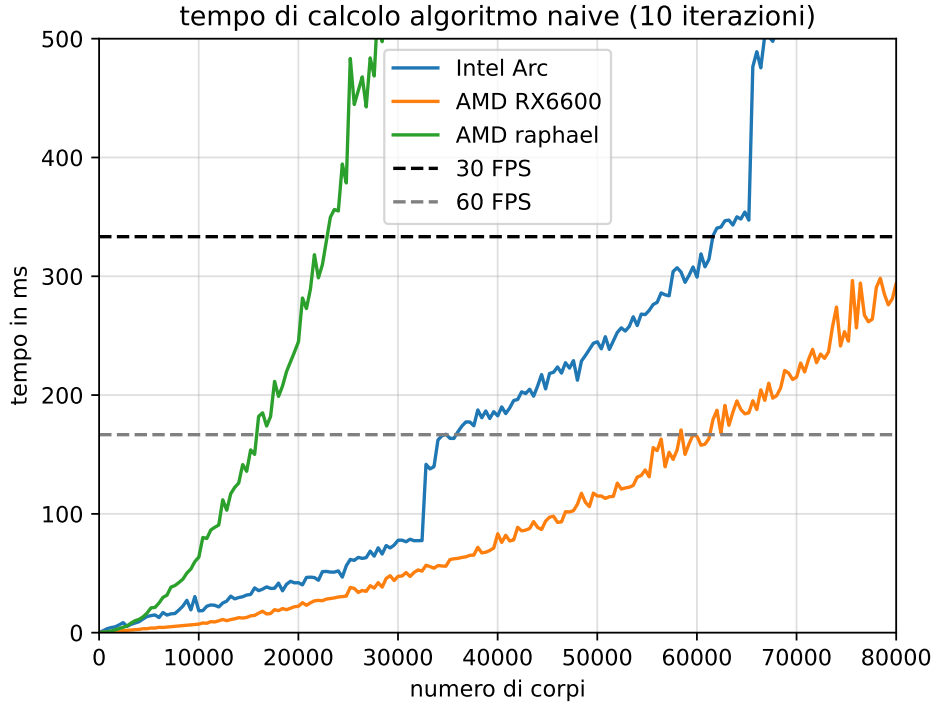


Immagine 6: Somma dei tempi dei kernel

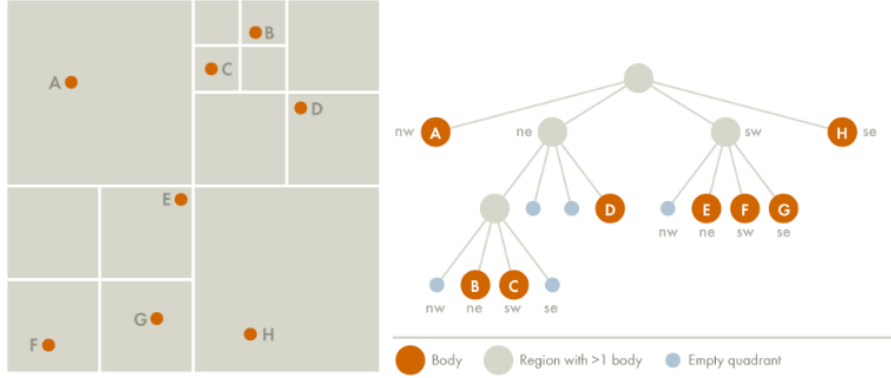
A 30 FPS, Raphael AMD può simulare 22800 corpi, l'Intel ARC fino a 61200 corpi e l'AMD RX6600 fino a 82800 corpi. A 60 FPS, Raphael non supera i 15610 corpi, l'Intel ARC i 34400 corpi e l'AMD RX6600 i 58000 corpi. La crescita quadratica dell'algoritmo naive (sia in termini di algoritmo, che in termini di accessi alla memoria globale) è il punto debole dell'algoritmo.

2.4 Un esperimento con la local memory

Proviamo a utilizzare la local memory per ridurre il numero di accessi in memoria. In via teorica, dovrebbe abbassare (non asintoticamente parlando) il numero di accessi alla memoria globale, che ribadiamo essere una delle principali fonti di overhead. Tuttavia, sappiamo anche che spesso la cache funziona meglio quando non si interferisce con la local memory: verifichiamo sperimentalmente la nostra ipotesi.

3 Simulazione approssimata Barnes-Hut su GPU

L'algoritmo Barnes-Hut permette di simulare sistemi a n corpi in maniera approssimata, con precisione arbitraria e complessità asintotica $\mathcal{O}(n \log n)$. Sfrutta una particolare struttura dati, ossia un quad-tree (figura 3) - un albero in cui ogni nodo ha quattro figli - per suddividere ricorsivamente lo spazio, memorizzare valori relativi al sistema e velocizzare il passaggio più oneroso della simulazione brute-force: il calcolo delle forze (o dell'accelerazione), notoriamente un passaggio a complessità $\mathcal{O}(n^2)$.



L'idea è la seguente: costruiamo un albero che descrive l'intero sistema, e invece di calcolare l'attrazione tra un corpo c e il resto dei corpi C , si visita l'albero, e per ogni altro nodo ci chiediamo "posso approssimare tutti i figli di questo nodo ad un unico corpo?". Lo **sviluppo in multipoli** ci dice che questo è possibile quando l'area che include questi corpi non è molto grande e quando la distanza da c è elevata: useremo un parametro θ soglia, che verrà confrontato proprio al rapporto tra le grandezze appena citate. Quando il criterio è rispettato, si accetta il nodo come approssimazione di tutti i corpi della regione associata, con posizione uguale al centro di massa, e massa uguale alla somma delle masse, altrimenti si continua la visita nell'albero. Su sistemi con molteplici regioni dense, ma molto distanti tra loro, l'algoritmo Barnes-Hut performa estremamente bene.

Tuttavia l'algoritmo paga l'overhead della costruzione del quad-tree, che richiede non pochi passaggi:

1. Definizione dell'area dell'intero sistema particellare, ossia la **bounding-box**.
2. Inserimento di ciascun corpo all'interno dell'albero.
3. Propagazione nei nodi interni delle masse e dei centri di massa dei rispettivi figli.

Questi passaggi sono i più costosi dell'algoritmo, ma che offrono proprio il vantaggio asintotico. Trattiamo adesso come è stato possibile ottenere un vantaggio tramite la programmazione parallela su GPU.

3.1 I kernel fondamentali

Calcolo dei margini della bounding-box I primi kernel sono quelli dedicati alla definizione della dimensione della bounding-box (ossia della radice del quad-tree). Usiamo una riduzione parallela per trovare il minimo e il massimo dell'intero sistema particellare, di ciascuna delle due dimensioni. In questo modo, sarà possibile costruire un cubo che contiene l'intero sistema, per ciascun istante di tempo t .

Pseudo-codice 6 Kernel per la riduzione parallela (ricerca del minimo o massimo)

Require: array A , array B , dimensione dell'input n , array $BoundingBox$, flag per l'ultima riduzione

```

1:  $j \leftarrow$  id globale del work-item
2:  $i \leftarrow 2j$ 
3: if  $i > n$  then return            $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
4: end if
5:  $val \leftarrow A[i]$ 
6: if  $i + 1 < n$  then
7:    $val \leftarrow \min(val, A[i + 1])$             $\triangleright \max(val, A[i + 1])$  nella ricerca del massimo
8: end if
9: if è l'ultima riduzione then
10:   $BoundingBox[MIN\_INDEX] \leftarrow val$             $\triangleright BoundingBox[MAX\_INDEX]$  nella ricerca del
11: end if
12:  $B[j] \leftarrow val$ 

```

Questo kernel sarà lanciato fino a quando il minimo (massimo) si troverà alla posizione 0 dell'ultimo array usato. Nel codice host, abbiamo deciso di usare due array, scambiando ogni volta l'array di input e quello d'output (una sorta di ping-pong tra gli array).

Costruzione del quad-tree Il secondo kernel che presentiamo, è quello relativo alla **costruzione del quad-tree**. Prima di presentare il kernel, definiamo in cosa consiste il nostro quad-tree in termini di strutture dati. Necessitiamo infatti un modo per linearizzare un quad-tree in un array (sia per le performance, che per le capacità)

```
int cell_children = array dei figli di ciascuna cella. 4 volte più lungo degli altri;
float2 cell_center = array delle posizioni (x,y) dei centri delle celle;
float cell_half_size = array di metà lunghezza del lato delle celle;
float2 cell_center_of_mass_mem = centro di massa della cella (basato sui figli);
float cell_mass_mem = massa della cella (somma delle masse dei figli);
```

La gestione degli indici in relazione ai figli è banale. In [?] viene fornito un esempio:

```

/-----|-----|
|0| |1|2|3|4| |5|6|7|8|9|10|11|12|13|...
\__|_____|

```

Nel nostro caso, nell'array dedicato ai figli, i 4 figli della radice si avranno in posizione 0,1,2,3. La profondità di un indice i nell'albero può inoltre essere determinata da $\lceil \log_4(i) \rceil + 1$. A parità di profondità, l'ordinamento dei quattro nodi rimane a nostra discrezione. Stabiliamo la seguente suddivisione, rispetto alla suddivisione dei quadranti:

```

/* CONVENZIONE:
   10 11
   00 01
*/
inline int get_quadrant(float2 pos, float2 center) {
    int right = pos.x >= center.x;
    int top = pos.y >= center.y;
    return (top << 1) | right;
}
```

In questo modo, accedere ai quattro figli di un nodo i consisterà nel calcolo $4i + j$, con $j = 0$ per il figlio che indica il quadrante in basso a sinistra, $j = 1$ per il quadrante in basso a destra, $j = 2$ per il quadrante in alto a sinistra e $j = 3$ per il quadrante in alto a destra.

In letteratura non si trovano esempi di algoritmi di costruzione di quad-tree paralleli su GPU senza ausilio delle funzioni atomiche. Usiamo quindi un approccio seriale, mantenendo tuttavia i dati su GPU, per risparmiare sullo scambio dati CPU-GPU. Ci rifaremo quindi alla costruzione del quad-tree dell'algoritmo originale, ossia.

1. Se il nodo corrente c non contiene un corpo, inseriamo il corpo b in c .
2. Se il nodo c è un nodo interno, inseriamo il corpo b

3.2 L'algoritmo complessivo

3.3 Valutazione delle performance

